

复杂建筑火灾救援疏散仿真 数学模型

1. 几何建模 (Geometric Modeling)

为精确描述搜救场景，我们构建了一个两层嵌套的 **3D 混合图模型 (Hybrid 3D Graph Model)**，该模型由一个宏观拓扑图和一个微观网格图组成。

1.1. 宏观拓扑图 (Macro-Level Graph)

宏观层是一个三维有权图 $G = (V, E)$ ，用于描述建筑的宏观连通性。

1. 节点集合 (V): 节点 $v \in V$ 是一个四元组 $v = (id, pos, z_{id}, type)$ ，其中：

- id : 节点的唯一标识符 (例如 `Apt4_Door`)。
- pos : 节点的三维空间坐标 $[x, y, z]$ 。其中，一层 $z = 0$ ，二层 $z = H_{\text{floor}}$ (例如 2.5m)。
- z_{id} : 节点所属的功能区域ID (`zone_id`)，用于关联动态环境模型。
- $type$: 节点类型, $type \in \{ \text{Entry}, \text{Room}, \text{Door}, \text{Transition} \}$ 。
 - Entry** (出入口): 响应者 R_i 的起始点 S_i 和终点 E_i 。 $S_i, E_i \subset V_{\text{Entry}}$ 。
 - Room** (房间质心): 功能区 (仓库、商铺、公寓) 的逻辑中心，用于关联区域烟雾。
 - Door** (连接点): 连接 **Room** 与 **Transition** 的物理门。
 - Transition** (过渡点): 楼梯 (`Stairs`) 和走廊 (`Hallway`) 的逻辑节点。

2. 边集合 (E): 边 $e \in E$ 代表两个节点 v_a, v_b 之间的物理连通性。

- 边权重 $w(e)$:** 边的权重不是静态距离，而是动态计算的**通行时间** $T(e)$ ，该时间取决于响应者到达 v_a 的时刻 t_{arrival} 和边的类型。
- 楼梯边 (e_{stairs}):** 对于连接 $v_a(z = 0)$ 和 $v_b(z = H_{\text{floor}})$ 的楼梯边，其基础距离 $d(e)$ 定义为：

$$d(e_{\text{stairs}}) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} + |z_a - z_b|$$

1.2. 微观网格图 (Micro-Level Grid)

对于内部结构复杂 (含障碍物) 的区域 (即仓库)，我们引入一个 2D 离散网格 M 。

- 网格 M 是一个 $H \times W$ 的矩阵，由 $G(i, j)$ 索引，其中 i, j 通过分辨率 ρ (例如 0.25m) 与世界坐标 (x, y) 映射。
- 网格单元的值 $M(i, j)$ 定义为：
$$M(i, j) = \begin{cases} 0, & \text{可通行区域 (Open Space)} \\ 1, & \text{墙壁边界 (Wall)} \\ 2, & \text{内部障碍 (Obstacle)} \end{cases}$$
- 约束:** 所有在 M 上规划的微观路径 P_{micro} 必须满足 $\forall p \in P_{\text{micro}}, M(p_i, p_j) = 0$ 。

2. 动态环境建模 (Dynamic Environment Modeling)

环境的危险性 (烟雾) 和响应者的能力 (速度) 是随时间 t 变化的函数。

2.1. 可视距离衰减模型

在任意功能区 z 中，视觉半径 $R_z(t)$ 随时间 t 呈指数衰减：

$$R_z(t) = (R_{z,\max} - R_{\min}) \cdot e^{-\lambda_z t} + R_{\min}$$

其中：

- t ：当前时间（秒）。
- $R_{z,\max}$ ：区域 z 的初始最大可视距离（通常为其对角线长度）。
- $R_{\min}$ ：最小可视距离（例如 0.5m）。
- λ_z ：区域 z 的衰减系数，由该区域的临界安全时间 $T_{\text{crit},z}$ 决定：

$$\lambda_z = \frac{3}{T_{\text{crit},z}}$$

（注：仓库 $T_{\text{crit}} = 1800s$ ，公寓 $T_{\text{crit}} = 360s$ ）

2.2. 动态速度分段函数

响应者在 t 时刻位于区域 z 时的最大移动速度 $v(t, z)$ ，是 $R_z(t)$ 的分段函数：

$$v(t, z) = \begin{cases} v_{\text{base},k}, & R_z(t) \geq R_{\text{thresh}} \\ v_{\min} + (v_{\text{base},k} - v_{\min}) \cdot \frac{R_z(t) - R_{\min}}{R_{\text{thresh}} - R_{\min}}, & R_{\min} \leq R_z(t) < R_{\text{thresh}} \end{cases}$$

其中：

- R_{thresh} ：能见度阈值（例如 2.0m）。
- $v_{\min}$ ：烟雾中搜救人员的最小移动速度（0.2m/s）。
- $v_{\text{base},k}$ ：基础速度，由场景 k 决定：

$$v_{\text{base},k} = \begin{cases} 1.2 \text{ m/s}, & k = \text{平路 (Straight)} \\ 1.1 \text{ m/s}, & k = \text{楼梯 (Stairs)} \\ 1.0 \text{ m/s}, & k = \text{转弯 (Turn)} \end{cases}$$

2.3. 搜索效率 (Search Efficiency)

在执行“搜索”类任务（周界或弓字形扫描）时，如果能见度 $R_z(t)$ 低于安全阈值 R_{sweep} （例如 2.5m），引入效率惩罚因子 $\eta = 0.3$ 。有效搜寻速度 v_{eff} 为：

$$v_{\text{eff}}(t, z) = v(t, z) \cdot \eta \quad \text{if } R_z(t) < R_{\text{sweep}}$$

3. 目标函数与约束 (Objective Function & Constraints)

3.1. 目标函数

本仿真的目标是最小化**总救援时间** T_{total} ，定义为从响应者开始行动（ T_{delay} ）到**最后一名**响应者完成所有任务并**撤离**到安全出口 V_{Entry} 的绝对时间 T_{finish} ：

$$\min T_{\text{total}} = \left(\max_{i \in \{1, \dots, N\}} (T_{\text{finish}}^{(i)}) \right) - T_{\text{delay}}$$

其中：

- N ：响应者总数（N_RESPONDERS）。
- $T_{\text{finish}}^{(i)}$ ：第 i 个响应者完成所有任务并撤离的绝对时间。
- T_{delay} ：初始响应延迟（START_DELAY）。

3.2. 约束条件

1. **速度约束:** $\forall i, t, z$, 响应者 i 的速度 $v_i(t, z) \in [v_{\min}, v_{\text{base}, k}]$ 。
2. **宏观连通性约束:** $\forall P_{\text{macro}}$ (宏观路径), P_{macro} 必须是 $G = (V, E)$ 中的一条有效路径。
3. **微观连通性约束:** $\forall P_{\text{micro}}$ (微观路径), P_{micro} 中的所有点 $p = (x, y)$ 必须满足 $M(\lfloor y/\rho \rfloor, \lfloor x/\rho \rfloor) = 0$ 。
4. **扫楼约束:** 所有功能区 $z \in Z_{\text{rooms}}$ 必须被一个对应的搜索任务 (周界或弓字形) 完全覆盖。

4. 求解算法与任务分配 (Solution & Allocation)

4.1. 任务分配 (Task Allocation)

1. **任务队列 (Q):** 建立一个包含所有待执行任务的全局队列 $Q = \{q_1, \dots, q_m\}$ 。
2. **优先级 (P):** 任务 q_j 按优先级 $P(q_j)$ 降序排列。
 - P_{high} : Infant, Pregnant, WH Search 1/2 (高危人员/区域)
 - P_{med} : Search Apt 2/3, Search Shop 1/3 (普通扫楼)
 - P_{low} : WH Sweep Zone 1, WH Sweep Zone 2, ... (仓库并行扫描)
3. **分配策略 (Greedy):** 仿真按时间步推进, 在 t_{now} 时刻, 队列头的任务 q_1 被分配给“最快可用”的响应者 i^* :

$$i^* = \arg \min_{i \in \{1, \dots, N\}} (T_{\text{free}}^{(i)})$$

任务开始时间为 $T_{\text{start}} = \max(t_{\text{now}}, T_{\text{free}}^{(i^*)})$ 。

4.2. 混合路径规划 (Hybrid Path Planning)

响应者 i^* 执行任务 q_j 的总时间 T_{task} 是宏观路径和微观路径耗时之和。

1. **高层路径 (Dijkstra):**
 - **算法:** find_path_high_level_3d。
 - **路径:** $P_{\text{macro}} = S_i \rightarrow V_{\text{target_door}}$ 。
 - **成本:** 算法在 3D 图 G 上求解, 边的权重 $w(e)$ 是基于 $v(t, z)$ 动态计算的通行时间。
2. **低层路径 (A*):**
 - **算法:** find_path_A_star_warehouse。
 - **用途:** 用于仓库内的点对点寻路 (例如 $V_{\text{target_door}} \rightarrow P_{\text{target_grid}}$)。
 - **成本:** $f(n) = g(n) + h(n)$
 - $g(n)$: 从起点到节点 n 的累积动态时间 (基于 $v(t, z)$)。
 - $h(n)$: 启发式函数, 使用欧氏距离 $d(n, n_{\text{goal}})$ 和最大速度 $v_{\text{base}, \text{max}}$ 估算:
 $h(n) = d(n, n_{\text{goal}}) / v_{\text{base}, \text{max}}$ 。
3. **低层路径 (并行弓字形扫描):**
 - **算法:** calculate_warehouse_sweep_v11。
 - **分区:** 仓库网格 M 被垂直分割为 K ($N_{\text{WAREHOUSE_RESPONDERS}}$) 个区域 M_k 。
 - **执行:** 响应者 i 被分配 M_k , 他首先 A* 寻路到 M_k 的扫描起点。

- **A* 辅助扫描：** 算法在 M_k 内逐行 (Boustrophedon) 扫描。当扫描行遇到障碍物 ($M(i,j)=2$) 或需要换行时, 再次调用 **A* 算法** 进行“跳跃”连接, 以规划绕过障碍物的路径。
- **成本：** 总时间为 $T_{A^*_to_start} + \sum T_{linear_scan} + \sum T_{A^*_jump}$ 。